



32.1. Python: Seaborn

Beautiful visualizations, powerful insights, less code.

Previous Section:

 [32. Python: Matplotlib](#)

Contents:

[What is Seaborn?](#)

[Why Use Seaborn?](#)

[When Should You Use Seaborn?](#)

[Importing Seaborn](#)

[5. Common Plots in Seaborn](#)

[5.1. Relational Plots](#)

[5.2. Categorical Plots](#)

[5.3. Statistical Plots](#)

[5.4. Distribution Plots](#)

[Summary](#)

[References](#)

In the previous section, we explored Matplotlib, the foundation of data visualization in Python. While Matplotlib is extremely flexible and powerful, creating attractive statistical graphs can sometimes require a considerable amount of code. This is where **Seaborn** comes in.

Seaborn is a high-level visualization library built on top of Matplotlib that focuses on creating beautiful and informative statistical graphics with minimal effort. Instead of manually customizing every element of a graph, Seaborn

provides sensible defaults, attractive color palettes, and specialized statistical plotting functions.

In many cases, a graph that requires several lines of Matplotlib code can be created using a single Seaborn function.

What is Seaborn?

Seaborn is a powerful Python visualization library built on top of Matplotlib, designed for statistical graphics, optimized for working with DataFrames and structured datasets. It is easy to use while still providing advanced customization

Because Seaborn uses Matplotlib internally, anything learned from Matplotlib can still be applied when using Seaborn. Think of Seaborn as a smarter and more specialized layer built above Matplotlib.

Why Use Seaborn?

Seaborn is popular because it helps us:

- Visualize large datasets quickly
- Create attractive graphs with minimal code
- Discover trends and patterns
- Compare categories and distributions
- Explore relationships between variables
- Build business and analytical reports

Most importantly, Seaborn allows us to focus on understanding the data rather than spending time designing the graph.

When Should You Use Seaborn?

Seaborn is especially useful when:

- Working with Pandas DataFrames
- Performing Data Analysis

- Exploring Machine Learning datasets
- Building business reports
- Investigating trends and relationships
- Creating statistical visualizations

If Matplotlib is a general-purpose plotting tool, Seaborn is a data analyst's visualization toolkit.

Importing Seaborn

To use Seaborn:

```
import seaborn as sns
```

Since Seaborn is built on Matplotlib, we often import both libraries:

```
import seaborn as sns
import matplotlib.pyplot as plt
```

5. Common Plots in Seaborn

To make learning easier, we will group Seaborn visualizations into four major categories: Relational Plots; Categorical Plots; Statistical Plots; Distribution Plots

For the examples below, we will generate sample datasets using NumPy.

```
import numpy as np
import pandas as pd

np.random.seed(42)

df = pd.DataFrame({"x": np.arange(100),
                   "y": np.random.randint(0, 100, 100)})
```

Using NumPy-generated data creates more realistic datasets and better demonstrates Seaborn's capabilities.

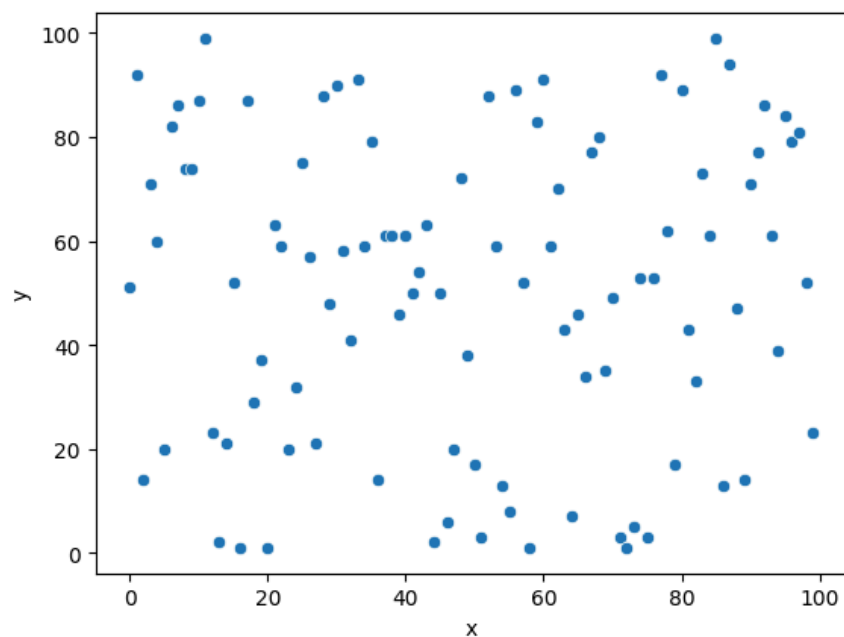
5.1. Relational Plots

Relational plots help us understand relationships between numerical variables. Questions such as: Does one variable increase with another? Is there a trend? Are variables correlated?, etc., can often be answered using relational plots.

- Scatter Plot displays individual observations. Useful for: Correlation analysis; Pattern detection; Outlier identification

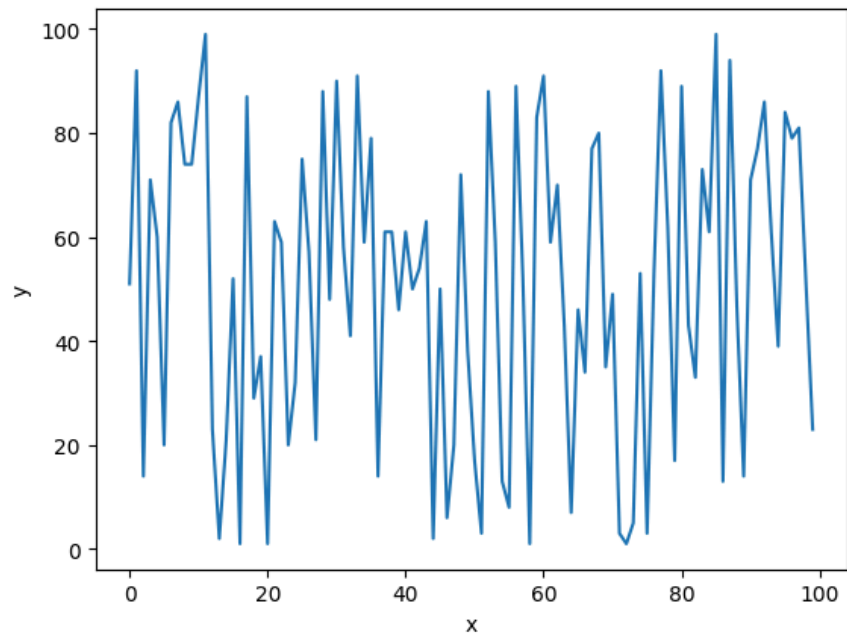
```
import seaborn as sns
```

```
sns.scatterplot(data=df, x="x", y="y")  
plt.show()
```



- Line Plot displays trends and changes over time. Common uses: Stock prices; Temperature records; Sales trends; Machine Learning metrics

```
sns.lineplot(data=df, x="x", y="y")  
plt.show()
```



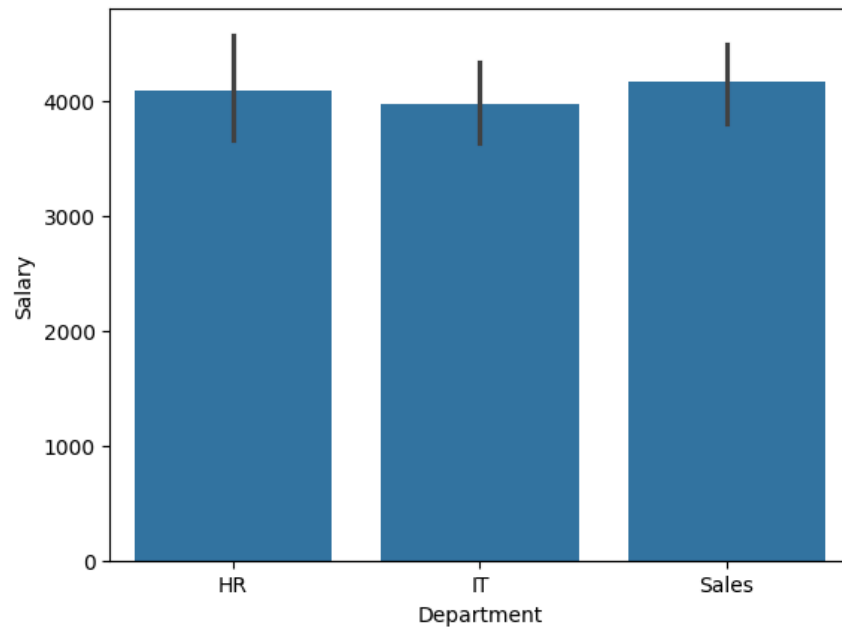
5.2. Categorical Plots

Categorical plots compare groups and categories. For demonstration:

```
df_cat = pd.DataFrame({"Department":
                        np.random.choice(["IT", "HR", "Sales"], 100),
                        "Salary": np.random.randint(2000,
                                                    6000, 100),
                        "Level": np.random.choice(["Boss",
                                                    "Leader", "Employee"], 100)})
```

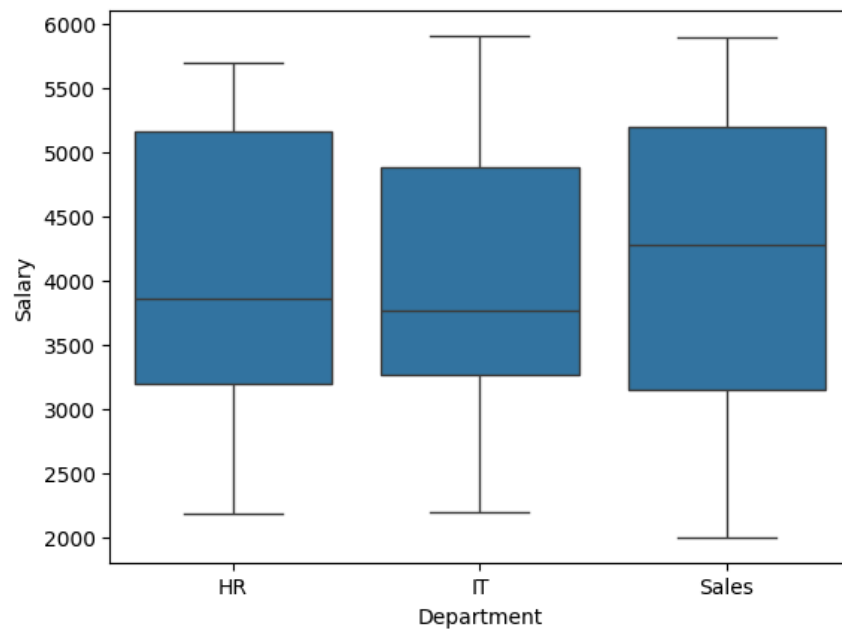
- Bar Plot shows average values across categories.

```
sns.barplot(data=df_cat, x="Department", y="Salary")
plt.show()
```



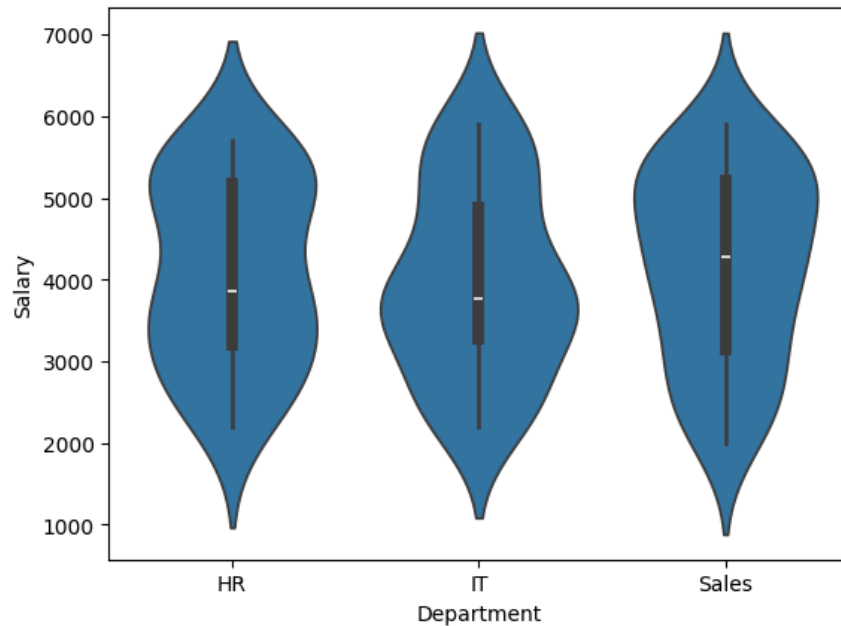
- Box Plot shows distribution and outliers.

```
sns.boxplot(data=df_cat, x="Department", y="Salary")  
plt.show()
```



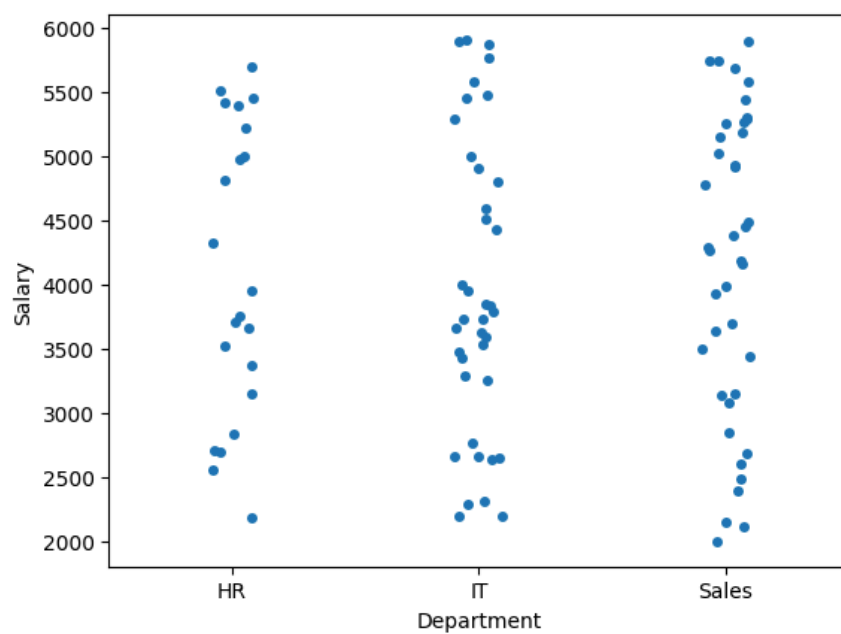
- Violin Plot combines a box plot with a distribution plot.

```
sns.violinplot(data=df_cat, x="Department", y="Salary")  
plt.show()
```



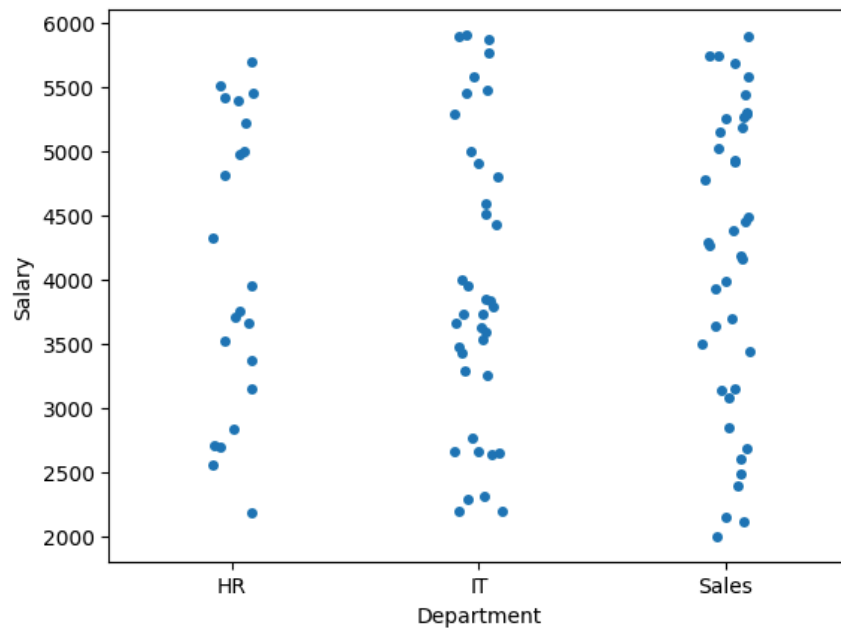
- Strip Plot displays individual observations.

```
sns.stripplot(data=df_cat, x="Department", y="Salary")  
plt.show()
```



- Swarm Plot is similar to strip plots but prevents overlap.

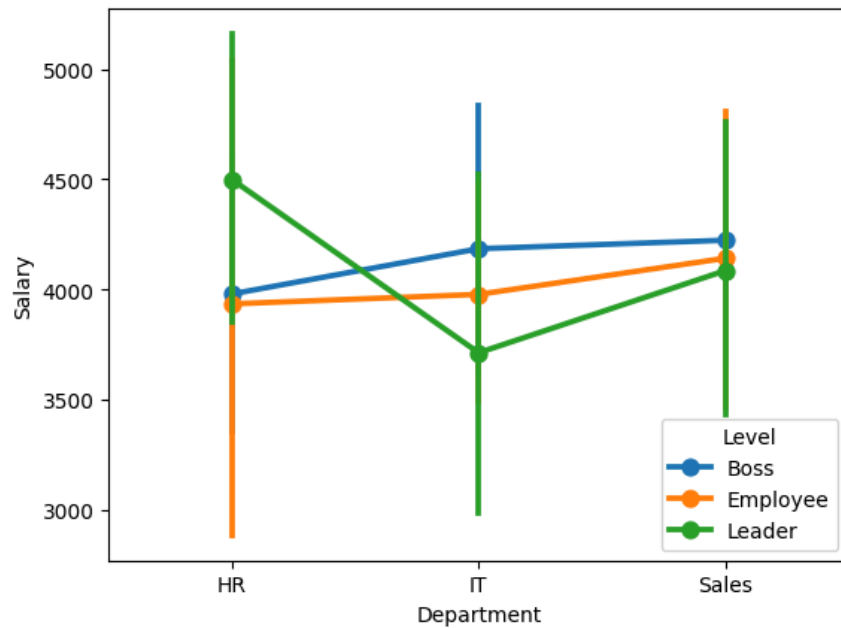
```
sns.swarmplot(data=df_cat, x="Department", y="Salary")  
plt.show()
```



- Point Plot displays category averages as points.

```
sns.pointplot(data=df_cat, x="Department", y="Salary", hue  
='Level')  
plt.show()
```

The **hue=** parameter assigns a different color to the data points and lines based on the categories in the **"Level"** column.

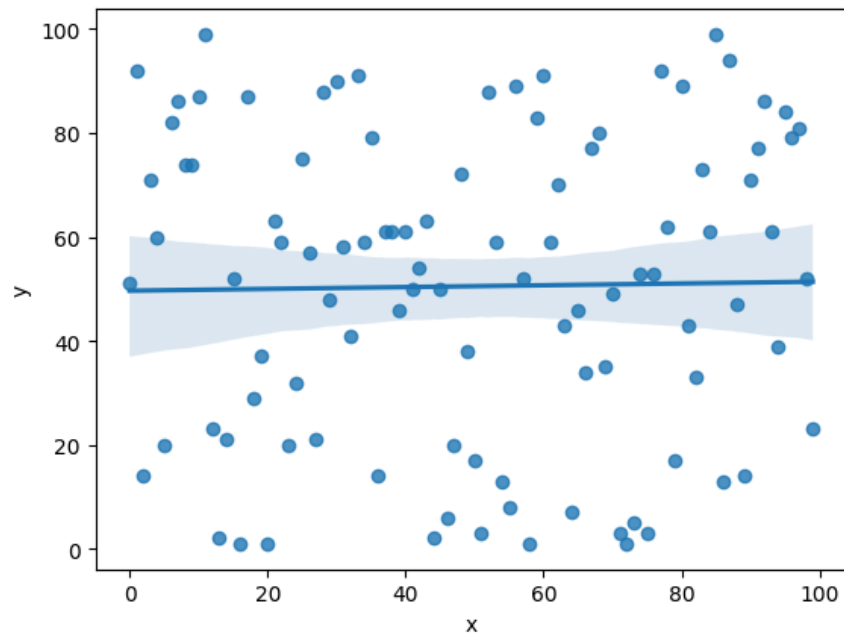


5.3. Statistical Plots

Statistical plots summarize data and reveal mathematical relationships.

- Regression Plot is a scatter plot with a regression line. Useful for: Trend analysis; Correlation studies; Predictive modeling

```
sns.regplot(data=df, x="x", y="y")  
plt.show()
```

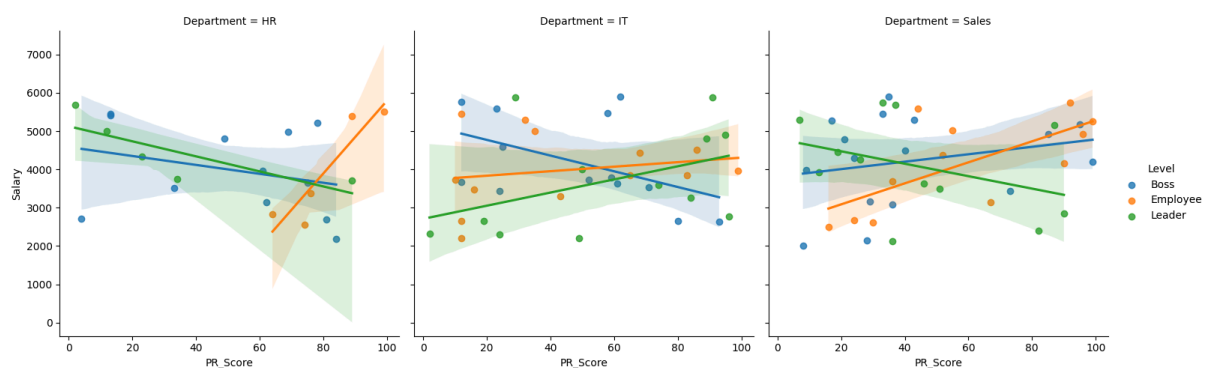


- LM Plot is an enhanced regression plot. It automatically creates a figure and regression model.

```
df_cat.insert(loc=3, column='PR_Score', value=np.random.randint(1, 100, 100))

sns.lmplot(data=df_cat, x="PR_Score", y="Salary", hue='Level', col='Department')
plt.show()
```

Add a new column specifically for this plot

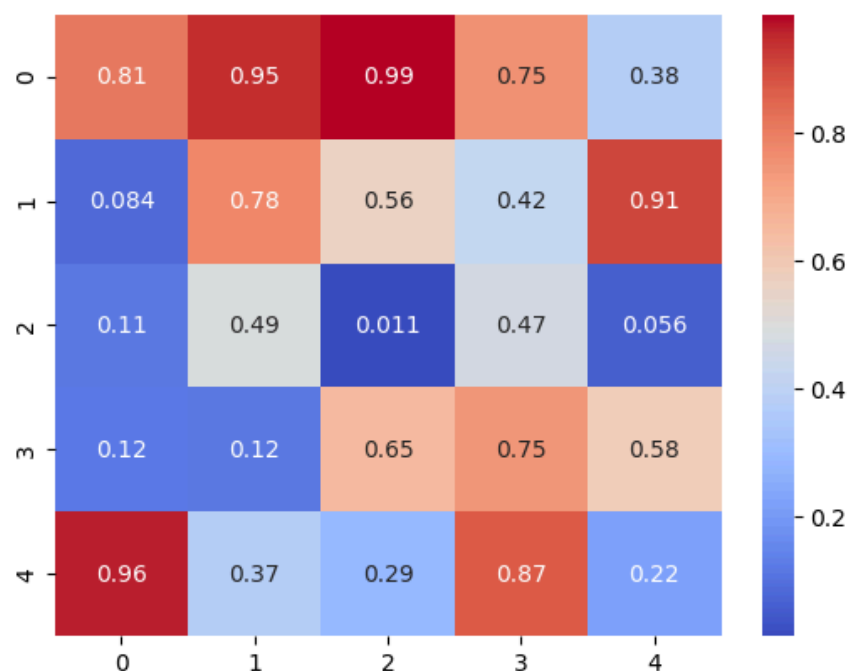


- Heatmap is one of Seaborn's most famous visualizations. Heatmaps are commonly used for: Correlation matrices; Confusion matrices; Feature analysis

```
import numpy as np

matrix = np.random.rand(5, 5)

sns.heatmap(matrix, annot=True, cmap="coolwarm")
plt.show()
```



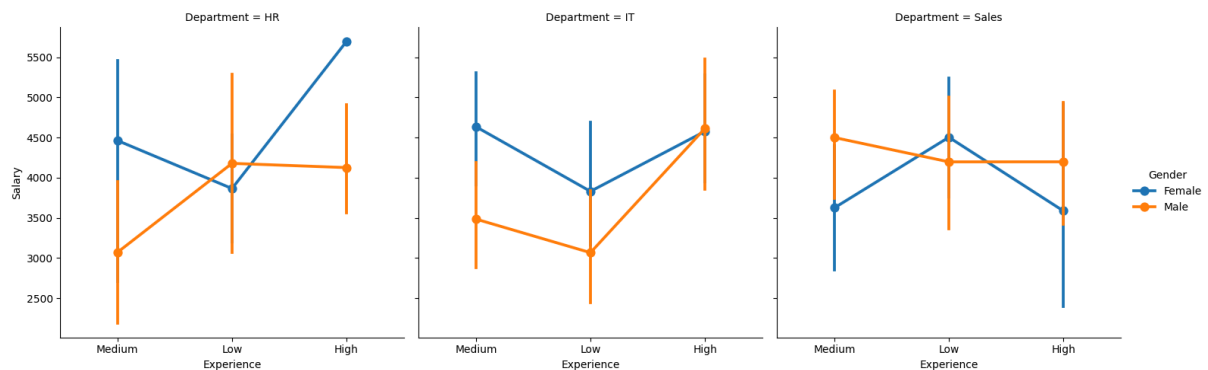
- `Catplot` is a flexible interface for categorical visualizations. Changing the `kind` parameter allows many different chart styles.

```
# Add Gender column
df_cat.insert(loc=2, column='Gender',
              value=np.random.choice(["Male", "Female"], 10
0))

# Add Experience Level (Low, Medium, High)
df_cat.insert(loc=3, column='Experience',
              value=np.random.choice(["Low", "Medium", "High"], 100))

sns.catplot(data=df_cat, x="Experience", y="Salary", kind
="point",
            hue='Gender', col='Department')
```

Add some new columns specifically for this plot

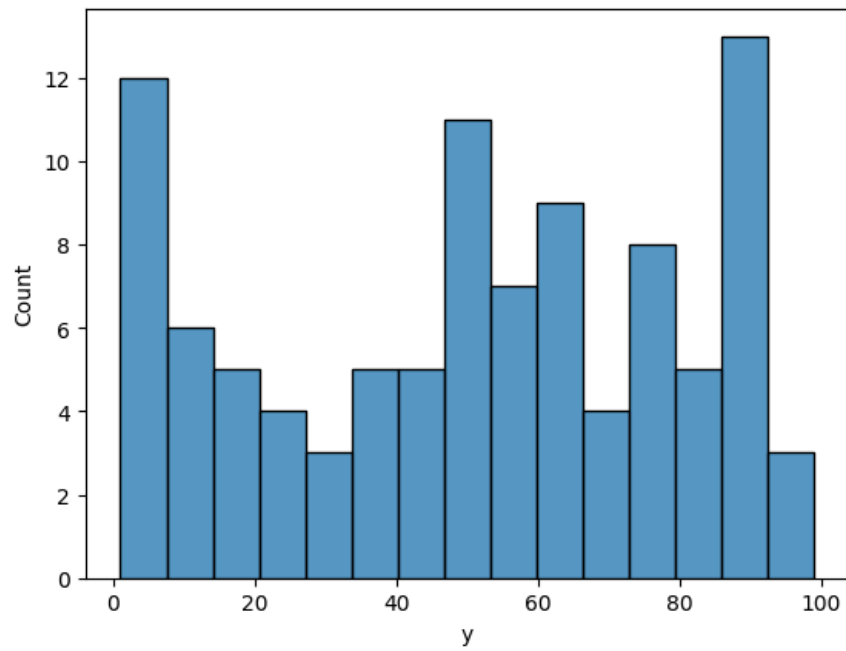


5.4. Distribution Plots

Distribution plots help us understand how data is spread.

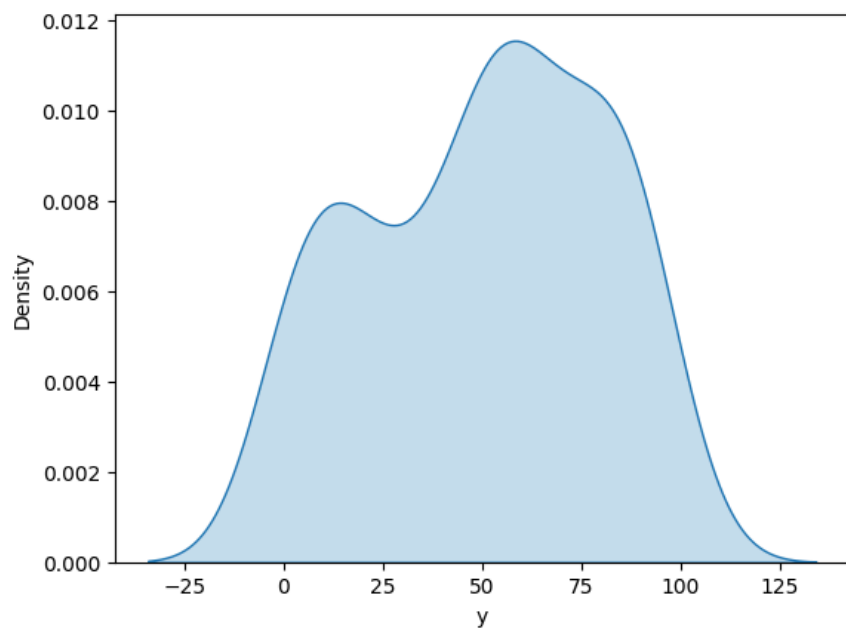
- Histogram shows frequency distributions.

```
sns.histplot(df["y"], bins=15)
plt.show()
```



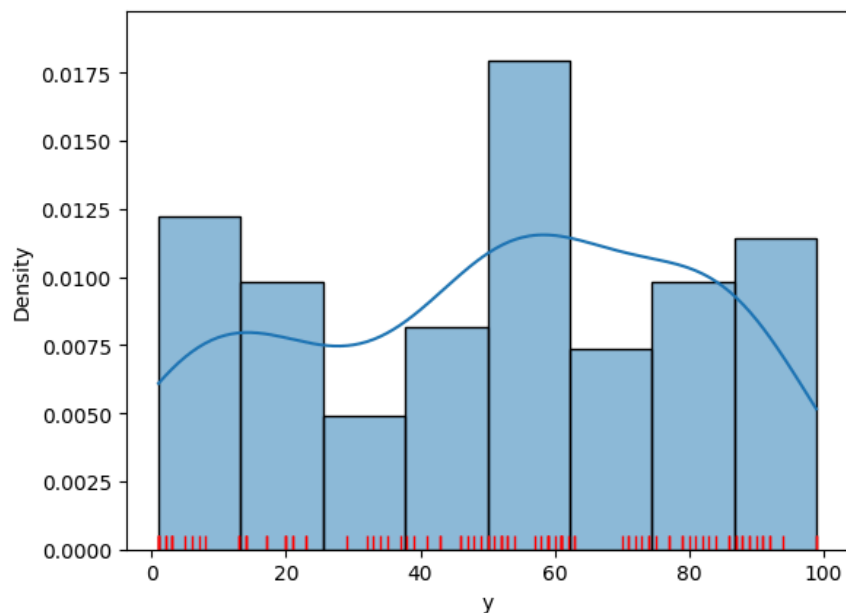
- KDE Plot displays a smooth probability density curve. This is often viewed as a smoother version of a histogram.

```
sns.kdeplot(df["y"], fill=True)  
plt.show()
```



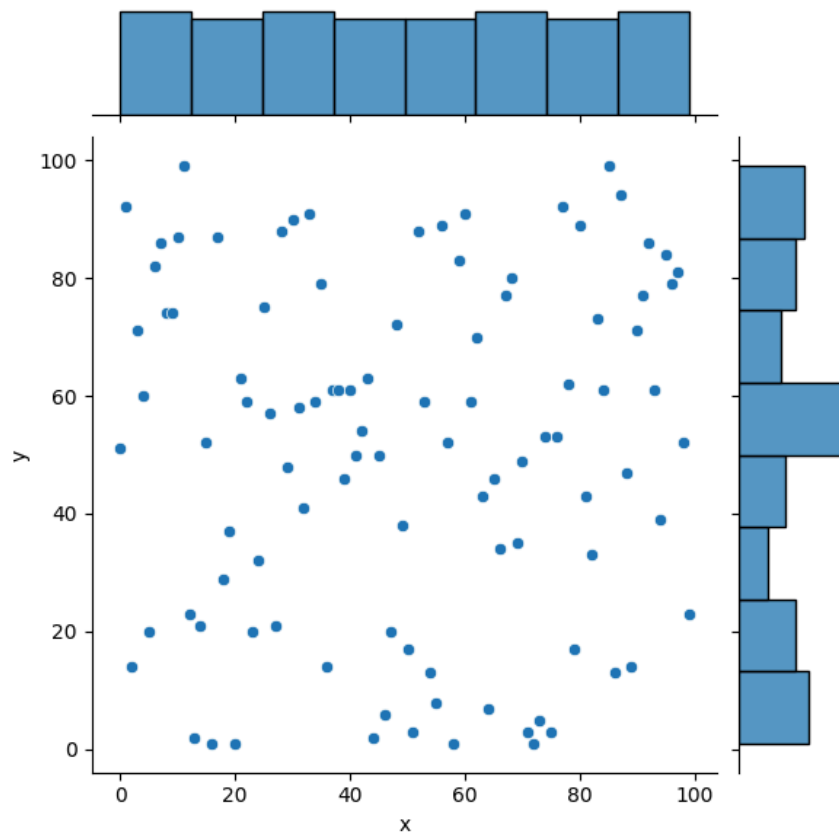
- Rug Plot displays every observation as a small tick mark. Useful for seeing exact data locations.

```
# Draw Histogram with KDE line alongside rugplot
sns.histplot(df["y"], kde=True, stat="density", linewidth=
1)
sns.rugplot(df["y"], color='red')
plt.show()
```



- Joint Plot combines multiple visualizations into one figure. A Joint Plot typically includes: Scatter Plot; Histogram of x ; Histogram of y , making it a powerful exploratory analysis tool.

```
sns.jointplot(data=df, x="x", y="y", kind="scatter")
plt.show()
```



Summary

Seaborn extends Matplotlib by providing cleaner syntax, better default styling, and powerful statistical visualization capabilities.

In this chapter, we explored four major groups of Seaborn plots:

Category	Common Functions
Relational	<code>scatterplot()</code> , <code>lineplot()</code>
Categorical	<code>barplot()</code> , <code>boxplot()</code> , <code>violinplot()</code> , <code>stripplot()</code> , <code>swarmplot()</code> , <code>pointplot()</code>
Statistical	<code>regplot()</code> , <code>lmplot()</code> , <code>heatmap()</code> , <code>catplot()</code>
Distribution	<code>histplot()</code> , <code>kdeplot()</code> , <code>rugplot()</code> , <code>jointplot()</code>

Compared to Matplotlib, Seaborn often produces more attractive visualizations with significantly less code, especially when working with Pandas DataFrames and statistical datasets.

Together, **Matplotlib** and **Seaborn** form the backbone of data visualization in **Python**, providing everything from simple line charts to advanced analytical dashboards used in Data Science, Machine Learning, and Business Analytics.

References

<https://seaborn.pydata.org/>

<https://www.geeksforgeeks.org/python/introduction-to-seaborn-python/>

Hand-On Exercises:

[`</>` Ex 4. Python Exercises](#)

Next Section:

[`</>` 33. Python: PyGame](#)